CSI4001 – Natural Language Processing and Computational Linguistics

Name: Sai Pravali Motamarri

Reg No: 23MIC0086

3. Investigate about the Embedding used profoundly in NLP applications. How are they built? What are baselines? Are embeddings available for our Indian regional languages - Name some and demonstrate a few things that they can do.

1. Introduction: What Are Embeddings and Why They Matter

One of the basic issues that NLP systems runs into is that computers don't understand words, they understand numbers. The earliest fix for this problem was creating one-hot encoding of each word which contains only zeros, with a single one in the very spot where it is positioned in the vocabulary. The flaw in this reasoning is evident if you take a moment to think about it: with such an encoding all the words have no relation to each other. The distance between "cat" and "dog" is the same as distance to "bureaucracy." Such encoding system does not take into account any meaning at all.

To fix this, embeddings were created. The basic idea is that instead of using a sparse encoding with one entry that is not zero, each word gets a dense vector, usually between 100 and 1000 dimensions, that is trained on the actual usage of words in tons of texts. The words that have similar usage will stay close in the vector space. This is the basis for almost every modern NLP application, whether it be classification, translation, question answering or named entity recognition.

2. How Embeddings Are Actually Built

Comparatively new methods of embedding can be classified into the three major categories highlighted below.

Count-based methods were the earliest approach. They entail using a co-occurrence matrix to collect the data about how often the words occurred in a corpus, and then reweighted, often using measures such as Positive Pointwise Mutual Information (PPMI) to reduce the influence of very common but uninformative words like “the” and “of”. Since this matrix is huge and sparse, techniques such as Singular Value Decomposition are used to compress it into something more manageable. This is essentially what Latent Semantic Analysis does.

Prediction-based (static) embeddings replaced count-based methods as the dominant approach through models that use self-supervised prediction as their training objective rather than direct matrix factorization:

- **Word2Vec** (Mikolov et al.) introduced two architectures called Continuous Bag of Words (CBOW) that predicts a target word given the surrounding context and Skip-gram, which does the reverse and predicts the context words given the target word. Skip-gram tends to perform better on rare words.
- **GloVe** (Global Vectors), from Stanford, takes a slightly different angle. It combines the intuition of both count-based and prediction-based methods, enabling it to work with global word-word co-occurrence counts across the whole corpus rather than sliding a small context window across the text like Word2Vec.
- **fastText,** developed at Facebook AI Research**,** improves on Word2Vec by transforming each word as a bag of character n-grams rather than treating it as a single indivisible unit, a feature especially important for morphologically rich languages, like most Indian languages, because the model can build a reasonable vector for a word it has never seen just by combining the subword pieces it has seen before.

The drawback of the three methods is the static nature of the representations employed: meaning that the word "bank (river)" and "bank (financial institution)" shall share the same representation.

Contextual embeddings have addressed this concern by developing a method of defining a word's vector as a function of its surrounding context. ELMo is based on bidirectional

LSTMs and employs them to generate context-sensitive representations of words. This approach was soon outstripped by models based on transformers, and most importantly BERT, which uses the self-attention mechanism to create representations in which every word's vector depends on all surrounding words. Today, most serious NLP applications rely on contextual embeddings based on transformer neural networks, whereas static embeddings like Word2Vec and GloVe serve mostly as lightweight baselines or components in situations with limited resources.

3. Baselines

The word "Baseline" gets used in two related but slightly different ways in this field.
The first use refers to **baseline models**, which are the simpler forms of a new approach that is compared against to demonstrate improvement. Random vector initialization and one-hot/TF-IDF representations can be considered examples of trivial baselines. Meanwhile, Word2Vec and GloVe were cutting-edge technologies ten years ago ,but nowadays (in connection with the development of new models using contextual embeddings based on BERT) they act as baselines.
The second context involves the **baseline evaluation methods**, which are subdivided into two types:

- **Intrinsic evaluation** assesses embeddings in a direct manner based on their knowledge in language, without being tied to a certain application. Compelling examples for intrinsic evaluation are word similarity benchmarks (comparing cosine similarity of pairs to human similarity ratings). In addition, there are analogy tasks in which for instance, the well-known relationship "king − man + woman ≈ queen" is tested, and specifically tested are quality of relation among concepts having similar meanings.
- On the contrary, **Extrinsic evaluation** checks the effectiveness of embeddings in practice; hence it shows whether the efficiency of embeddings leads to actual performance improvement in text classification, named entity recognition or machine translation.

4. Embeddings for Indian Regional Languages

India has a total of 22 scheduled languages, in addition to a massive number of dialects, yet for years, the attention of NLP studies has been focused solely on English and a handful of other high-resource languages. Luckily, recent advances in this area have made it possible to pay attention to some of India's end number of languages and dialects thanks to the initiative of AI4Bharat, a project created by and hosted at IIT Madras.

- **AI4Bharat IndicNLP embeddings** (Kunchukuttan et al., a collaboration between IIT Madras, Microsoft India, and AI4Bharat) is among the initial resources for Indic NLP. The main highlight of this project is that it created a massive monolingual corpus for Indian languages and provided pretrained embeddings based on it, that was evaluated and found to contain 2.7 billion words in 10 Indian languages from 2 language families, while the pretrained word embedding resource was introduced alongside news category classification datasets specifically developed to evaluate the embeddings in terms of performance – it turned out that the Indic NLP embeddings are significantly better than other available pretrained embedding solutions. This corpus and embedding launch is widely recognized as one of the initial sources of good Indic NLP research work. arxiv
- **IndicBERT** was the next step in spreading Indian language representation into the modern world of contextual technology. It is a transformer model based on the BERT framework that was trained across 12 relevant Indian languages. It was created to provide contextual embeddings that can be used for various purposes: classification, NER, etc. There have been a few major releases of IndicBERT since then, during which the amount of input data and language coverage have been increased.
- An interesting example of the application of IndicBERT embeddings can be seen in **IndicNER**. It is adjusted to perform named entity recognition in 11 Indian languages which include Assamese, Bengali, Gujarati, Hindi, Kannada, Malayalam, Marathi, Oriya, Punjabi, Tamil and Telugu. Its performance has been evaluated against a human-annotated test set and some Indian NER datasets.
- **IndicTrans2** takes the same approach to machine translation. It is a model with 1 billion parameters trained on the Bharat Parallel Corpus Collection. It performs both English-to-Indic and Indic-to-English translation in 22 Indian languages. It solves the problem of different script writing for different Indian languages, which is quite the engineering task.

- Mentioning Google's MuRIL will be reasonable for comparison. The reason is that this model is developed based on the same multilingual embeddings concept with a focus on Indian languages and applying the same aims to represent low-resource Indian languages as well as it was done for English language.

5. Demonstration: What These Embeddings Can Do

The following few concrete examples would illustrate how these tools work:

- **Word similarity/analogy in Hindi:** The use of IndicNLP FastText vectors, search for the closest words to "राजा" (king) returns words associated with governance and kingship. The process of vector arithmetic that was shown in the English example "king - man + woman = queen" also works fairly well in the case of Hindi, even though the results differ somewhat because of variations regarding gender and formality encoding in both languages. However, these examples suggest that the embeddings are capable of capturing real semantic relations and not just superficial correlations of words.
- **Cross-lingual translation with IndicTrans2:** By taking the English sentence "The farmer sold his crops in the local market" and translating it into Tamil we get an illustration of how the model is able to keep the meaning of the sentence intact even though Tamil has an entirely different word order (subject-object-verb) and is mostly agglutinative.
- **Named entity recognition with IndicNER:** Given a Hindi sentence like "नरेंद्र मोदी ने दिल्ली में एक कार्यक्रम को संबोधित किया" (Narendra Modi addressed an event in Delhi), IndicNER correctly tags "नरेंद्र मोदी" as a PERSON entity and "दिल्ली" as a LOCATION entity, indicating that entity boundaries have been captured by the context of the sentence even with a different script and grammar compared to English language contexts.

6. Conclusion

The advancements in embeddings, from non-dynamic counting vectors to transformer encoding, have contributed greatly to developments in the field of NLP in the past decade. In the case of India, diverse linguistically, with 22 official languages and plenty of dialects, only

a relatively small number of people are connected to the English language NLP sources. However, some projects, such as AI4Bharat's IndicNLP corpus, IndicBERT, IndicNER, and IndicTrans2, have made an important step toward solutions since they show how to adapt and modify general embedding techniques used for English language to morphological complex low-resource languages.

References

- Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT 2019* (pp. 4171–4186). Association for Computational Linguistics. https://arxiv.org/abs/1810.04805
- Kunchukuttan, A., Kakwani, D., Golla, S., Gokul, N. C., Bhattacharyya, A., Khapra, M. M., & Kumar, P. (2020). AI4Bharat-IndicNLP corpus: Monolingual corpora and word embeddings for Indic languages. *arXiv preprint*, arXiv:2005.00085. https://arxiv.org/abs/2005.00085
- Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient estimation of word representations in vector space. *arXiv preprint*, arXiv:1301.3781. https://arxiv.org/abs/1301.3781
- Pennington, J., Socher, R., & Manning, C. D. (2014). GloVe: Global vectors for word representation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)* (pp. 1532–1543). https://nlp.stanford.edu/pubs/glove.pdf
- AI4Bharat. (n.d.). *IndicBERT: Multilingual language representation model.* IIT Madras. https://ai4bharat.iitm.ac.in/indic-bert
- AI4Bharat. (n.d.). *IndicTrans2: Neural machine translation for Indic languages.* IIT Madras. https://ai4bharat.iitm.ac.in/indic-trans2